

Autonomous Mass-Aware Sorting System

Yonghao Huo, Lisha Yuan, Han Wu, Zejun Xiong

University of California, Los Angeles

Los Angeles, CA, USA

{yonghh1, yuanlisha, hanwu0604, zejunxiong}@ucla.edu

Abstract—Warehouse and parcel-return facilities routinely handle objects whose mass is unknown until they reach a dedicated scale, which adds a process step and risks damage when heavy items are stacked on light ones. This work presents a simulated robotic sorting cell in which a UR5e manipulator picks an object from a conveyor, infers the object’s mass using only its own control signals, and places the object in the matching bin. No force/torque sensor or external scale is used. Four payload estimators are implemented and compared on a common interface: a single-joint PID-error method, an energy-balance (spring-sag) method, a generalized-momentum disturbance observer, and an inverse-dynamics residual method. Two trajectory-tracking controllers, a PD controller with gravity compensation and an inverse dynamics (computed-torque) controller, execute the pick-and-place motion, with the estimated mass fed forward as payload compensation. Across a 10 g to 3 kg sweep, the momentum observer is the most accurate estimator (0.9 % RMSE), and the inverse dynamics controller maintains the tracking and torque requirements over a wider range of motion speeds than the PD baseline. The recommended architecture combines inverse dynamics control with momentum-observer estimation.

Index Terms—payload mass estimation, momentum observer, inverse dynamics control, gravity compensation, robot manipulator, MuJoCo simulation

I. INTRODUCTION

E-commerce returns and mixed-parcel logistics confront robotic cells with streams of visually similar objects whose masses differ by orders of magnitude. In a conventional line, weighing and sorting are separate stations: an object is transported to a scale, measured, and then routed. Beyond the extra handling time, not knowing the mass during manipulation has direct consequences for control: a feedforward tuned for an empty gripper degrades when a heavy object is grasped, and stacking heavy objects onto light ones damages goods. Robotic manipulation studies in warehouse settings, such as the Amazon Picking Challenge, identify object-property uncertainty as one of the dominant failure sources in pick-and-place systems [1].

This project asks whether the weighing station can be eliminated entirely: can a torque-controlled manipulator weigh the object it is already holding, using only the signals its joint controller produces, and sort it accordingly? The system is a simulated UR5e arm that picks a cube from a conveyor, weighs it without any force/torque sensing, classifies the cube against a threshold, and places it into a light or heavy bin.

The task has two phases, each handled by a different component. In the *estimation phase*, the arm holds the grasped

cube at a weighing pose and a payload estimator infers its mass from the controller’s own signals, which sets the target bin (Section V-A). In the *transport phase*, a trajectory-tracking controller carries the cube to that bin while feeding the estimated mass forward as payload compensation (Section V-B). Because no single estimation principle is clearly best, we treat the problem as a case study: four estimators with different physical bases share one interface and are tested against two tracking controllers, both separately and as integrated pairs.

II. LITERATURE REVIEW

Estimating a grasped load’s inertial properties is a classical problem. Atkeson, An, and Hollerbach [2] identified all ten inertial parameters of a rigid load from wrist force/torque measurements during motion, using a formulation linear in the unknowns. We share the goal but impose two restrictions: no force/torque sensor anywhere, so the load is inferred from joint-level control signals alone, and only mass is estimated, since mass alone sets the sorting decision and the gravity feedforward.

The generalized-momentum disturbance observer originates in collision detection. De Luca and Mattone [3] built a momentum residual that converges to the external joint torque without acceleration measurements, for detecting unexpected contacts; Haddadin, De Luca, and Albu-Schäffer [4] survey two decades of this residual for collision detection, isolation, and identification. That work treats the external torque as a fault; we reuse the same observer to *measure* the payload’s weight, regressing the converged residual against a gravity model. Unlike the collision setting, this needs an empty-arm baseline subtraction to cancel systematic model error (Section V-A3).

On control, the two tracking controllers span the classical model-based range. Takegaki and Arimoto [5] proved that PD with gravity compensation globally stabilizes setpoint control of rigid manipulators, which grounds our PD+G baseline. The inverse dynamics (computed-torque) controller follows the standard formulation in Siciliano et al. [6], which also supplies the trapezoidal-velocity (LSPB) trajectory primitives. Slotine and Li [7] adapt unknown payload parameters online inside the tracking loop; we instead separate estimation (a weighing phase) from compensation (a feedforward term), giving up simultaneous adaptation for an explicit mass estimate the classifier needs anyway.

The simulation itself is built on the MuJoCo physics engine [8]. The UR5e and Robotiq 2F-85 models are taken

from the open-source MuJoCo Menagerie collection [9], and the analytical inverse kinematics follows the ROS-Industrial `ur_kinematics` formulation [10].

III. OBJECTIVES AND DESIGN REQUIREMENTS

The objectives of the controller and estimator design are:

- 1) **Estimation.** Estimate the payload mass accurately enough for reliable classification, without force/torque sensors, and characterize how four different estimation principles fail as the payload becomes light.
- 2) **Tracking.** Track the pick-and-place trajectory within stated accuracy and actuator limits as payload mass and motion speed vary.
- 3) **Integration.** Determine which controller-estimator pairs work best, and whether estimator accuracy depends on the tracking controller it runs under.

These objectives are quantified by four design requirements, referenced throughout the report:

- **C1 (tracking accuracy):** every joint position error stays within $\pm 2^\circ$ of its reference at all times. This bound keeps the end-effector well within the bin opening at the drop pose and guarantees grasp repeatability at the conveyor.
- **C2 (torque feasibility):** the commanded torque at each joint stays below 0.95 times the actuator limits

$$\tau_{\max} = [150, 150, 150, 28, 28, 28] \text{ N} \cdot \text{m}, \quad (1)$$

matching the UR5e model's actuator ranges with a 5% safety margin so the controller never saturates.

- **E1 (estimation accuracy):** absolute relative estimation error below 5% over the sorting range. With the classification threshold at 0.35 kg, a 5% error can misclassify only objects already within 5% of the boundary.
- **E2 (estimation range):** the estimator must operate from 10 g to 3 kg, covering three orders of magnitude in payload mass.

IV. SCENE SETUP

A. Robot Arm Selection

The UR5e is a 6-DOF collaborative robot from Universal Robots, with a reach of roughly 850 mm and a 3–5 kg payload rating that matches the parcel-sorting application. Its six degrees of freedom are enough for orientation-aware top-down grasps across the workspace while keeping the dynamics simple enough to model and analyze, and its kinematics are well documented, which makes it a common choice for simulation-based manipulation studies.

The simulation uses the torque-actuated UR5e description from the MuJoCo Menagerie [9], with a Robotiq 2F-85 two-finger gripper (also from [9]) attached at the flange. The torque-actuated model matters for two reasons: requirement C2 is stated against its actuator limits (1), and all controllers in this work command joint torques directly rather than relying on a black-box position interface. An early design iteration locked two wrist joints and controlled only four joints to simplify the problem; this was abandoned because the locked-joint

configuration removed too much gripper-orientation freedom for reliable top-down grasps across the workspace. The final system actively controls all six joints, which gives orientation-aware inverse kinematics and a consistent vertical pinch orientation at grasp, weigh, and release.

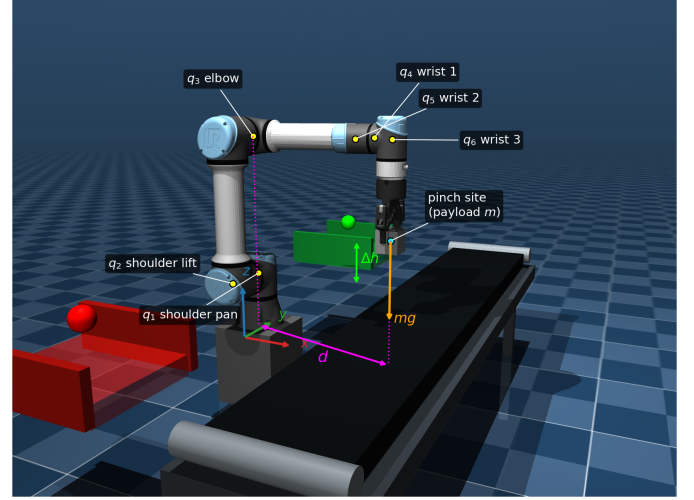


Fig. 1. Kinematic diagram at the weigh pose $q_{\text{weigh}} = [0, -90, -90, -90, 90, 0]^\circ$. The world frame (x, y, z) sits at the robot base; $q_1 \dots q_6$ label the six revolute joints referenced by all per-joint error plots. The payload of mass m hangs at the gripper pinch site and exerts the weight mg ; d is the horizontal moment arm from the elbow axis to the payload used by the sPID estimator (Section V-A1), and Δh is the vertical end-effector sag read by the energy-balance estimator (Section V-A2).

B. Task Scenario and Objective

The scene (Fig. 2) is a simplified conveyor sorting cell: a conveyor delivers a cubic block (a stand-in for a box or package) into the arm's workspace, and two bins sit on opposite sides for light and heavy objects. The arm picks the block, moves it to a fixed weighing pose, estimates its mass, and places it in the light or heavy bin according to a fixed threshold. Using simple cubes and clearly marked bins keeps the focus on the mass-estimation and sorting logic rather than on grasping or perception detail.

In the implemented scene, the payload cube measures 6 cm on a side and its mass is set per trial (10 g to 3 kg). Fig. 1 defines the world frame, the joint angles $q_1 \dots q_6$, and the geometric quantities used by the estimators; all per-joint plots in Section VI refer to these labels. The physics step is 2 ms and the controllers run at the same 500 Hz rate. The scene exposes joint position, joint velocity, and per-joint applied-torque sensors; no force/torque or tactile sensing exists anywhere in the model. Perception reads the cube pose from simulation ground truth (a camera-based backend is a planned substitution and does not affect the control problem studied here).

Two pipelines run the same estimators. The *weighing pipeline* uses a finite-state machine (search, grasp, weigh, classify, place, home) with a PD+gravity controller; the cube is physically grasped and weighed at a stationary hold. The

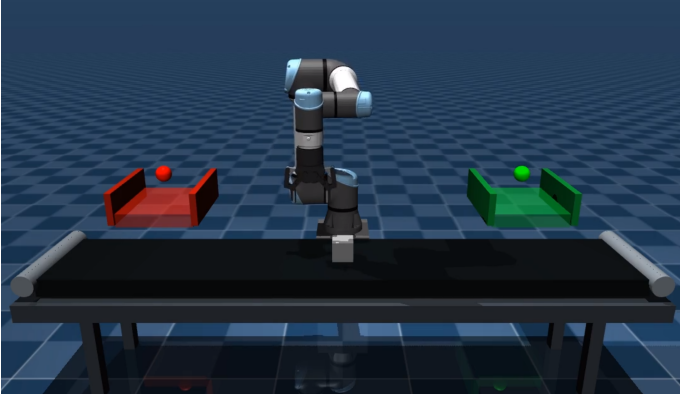


Fig. 2. Simulation environment setup for the mass-aware sorting task. The scene includes a conveyor belt, simplified box objects, a predefined weighing pose, and two target bins for light and heavy object classification.

tracking pipeline runs smooth LSPB trajectories with the two controllers of Section V-B, modeling the payload as a pure force $[0, 0, -mg]$ at the pinch site with the cube kinematically pinned to the end-effector. The distinction matters: the grasp-based pipeline passes unmodeled gripper-linkage torques to the arm, while the force-based one is ideal by construction (Section VI-D).

V. METHODS

A. Payload Estimators

All estimators start from the rigid-body equation of motion of the $n = 6$ joint arm with an unknown payload acting as an external joint torque τ_{ext} :

$$B(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + \tau_{\text{ext}}, \quad (2)$$

where $B(q)$ is the joint-space inertia matrix, $C(q, \dot{q})\dot{q}$ collects Coriolis and centrifugal torques, $g(q)$ is the gravity torque, and τ is the commanded joint torque [6]. For a point payload of mass m at the end-effector, the external torque is the pull-back of the payload weight through the linear Jacobian $J_v(q) \in \mathbb{R}^{3 \times 6}$ of the pinch site:

$$\tau_{\text{ext}} = J_v^T(q) \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} = -mg J_z(q), \quad (3)$$

with J_z denoting the third row of J_v . Each estimator is a different way of isolating the single unknown m in (3) from signals that the controller already has. The four estimators consume an identical per-tick observation (joint state, commanded and measured torque, gravity bias, inertia matrix, end-effector Jacobian and height), so they are interchangeable behind one interface and differ only in which channels they read.

Three of the four estimators require a one-time *empty-arm calibration*: with nothing in the gripper, the arm holds the weigh pose under exactly the gains and gravity-compensation configuration that will be used while weighing, and the relevant baseline signal is averaged over a three-second hold. Subtracting this baseline at estimate time cancels every

configuration-constant error (gravity-model mismatch, gripper-linkage spring torques, integration offsets), at the cost of an extra calibration pass whenever the weigh pose changes.

The following assumptions are made throughout. (i) The arm and payload are rigid bodies, and the simulator's inertial model is exact, so $g(q)$ and $B(q)$ queried from the model represent the true plant. (ii) The payload is a point mass at the gripper pinch site; center-of-mass offset and payload inertia are not identified. (iii) Joint positions and velocities are available noise-free at 500 Hz, and the commanded torque is realized exactly within the actuator limits (1). (iv) No force/torque sensing is available. (v) The sPID and energy-balance estimators additionally assume a quasi-static weighing phase (the arm at rest); the momentum observer and inverse-dynamics estimators do not. (vi) In the tracking pipeline, the payload acts as a pure gravity force at the end-effector; its inertial reaction during acceleration is not simulated.

1) *Single-Joint PID-Error Estimator (sPID)*: The sPID method turns one joint into a scale. At a fixed pose, a joint's torque must balance the gravity load of everything below it; normally the controller's gravity feedforward supplies this and the feedback does nothing, hiding the payload signal. The estimator therefore *disables gravity compensation at one measurement joint* (the elbow, q_3), so the elbow's feedback must hold the distal arm against gravity itself and its commanded torque becomes the measurement. At static equilibrium,

$$\bar{\tau}_{\text{cmd}} - \tau_{\text{ss,empty}} = mg d(q), \quad (4)$$

where $\bar{\tau}_{\text{cmd}}$ is the elbow torque command averaged over the one-second weigh hold, $\tau_{\text{ss,empty}}$ is the calibrated empty-arm baseline (which absorbs the arm's own distal weight and all static model error), and $d(q)$ is the horizontal moment arm from the elbow axis to the payload (Fig. 1). The moment arm is computed from the kinematic model: for a revolute joint with axis $\hat{a}$ and payload position $r(q)$ relative to the joint,

$$d(q) = (r(q) \times \hat{g}) \cdot \hat{a}, \quad \hat{g} = (0, 0, -1), \quad (5)$$

evaluated live from the end-effector Jacobian at the weigh pose ($d \approx 0.46$ m). Solving (4) for the mass gives the estimator

$$\hat{m} = \frac{\bar{\tau}_{\text{cmd}} - \tau_{\text{ss,empty}}}{gd}. \quad (6)$$

The denominator of (6) is purely geometric, fixed by (5); the numerator is the only quantity measured at run time.

Using one joint keeps the math simple (several uncompensated joints would couple the payload torque across the chain) and places the measurement near the payload, away from proximal-link model error. One practical issue: at the UR5e's stiff gains ($k_p \approx 2000$ N·m/rad) the uncompensated joint enters a limit cycle whose torque varies by tens of N·m, burying the sub-1 N·m signal of a 100 g cube. Softening that joint to $k_p = 200$, $k_d = 80$ during weighing removes the limit cycle without changing the steady-state torque: the joint settles a few degrees from its reference rather than a few millidegrees, and after one second of averaging the standard error is far below even the lightest cube's signal.

2) *Energy-Balance (Spring-Sag) Estimator*: This estimator was called the “Lyapunov” method in our presentation; we use the clearer name *energy-balance*, since the derivation rests on a static energy balance, not a Lyapunov stability argument. It reads positions, not torques. With the integral gain zero and gravity compensation on, each joint behaves as a torsional spring of stiffness $k_{p,j}$ about its reference: an empty arm rests at the reference, and a payload sags every joint until the spring torques balance its weight. Comparing the loaded rest pose q_{loaded} with the calibrated empty pose q_{empty} , the springs store extra elastic energy

$$\Delta E_{\text{spring}} = \frac{1}{2} \sum_{j=1}^6 k_{p,j} \left[(q_{\text{loaded},j} - q_{\text{ref},j})^2 - (q_{\text{empty},j} - q_{\text{ref},j})^2 \right], \quad (7)$$

while gravity does work $W_{\text{grav}} = mg \Delta h$ on the payload as the end-effector drops by $\Delta h = z_{\text{empty}} - z_{\text{loaded}}$ (Fig. 1). Equating W_{grav} with (7) overestimates the spring’s share, because it ignores the damper. In a rest-to-rest step of a spring-damper pair under constant load, the damper dissipates exactly as much as the spring stores: for one joint with disturbance torque F , the sag is $\delta = F/k_p$, gravity does $F\delta = F^2/k_p$ of work, and the spring stores only $\frac{1}{2}k_p\delta^2 = F^2/(2k_p)$, leaving the other half to the damper. The correct balance is therefore

$$W_{\text{grav}} = 2 \Delta E_{\text{spring}}, \quad (8)$$

independent of the damping level, and the mass estimate is

$$\hat{m} = \frac{2 \Delta E_{\text{spring}}}{g \Delta h}. \quad (9)$$

No moment arm appears anywhere: the geometry is carried implicitly by the kinematic chain.

Two controller overrides are essential. The integral gain must be zero; otherwise it winds up until the error vanishes, the springs relax, and the signal disappears. The proportional gains are softened tenfold during weighing, with k_d halved to preserve the damping regime and with it the rest-to-rest assumption behind (8). At full stiffness a 200 g cube sags the end-effector by under a millimeter, below the numerical noise, whereas the softened arm sags by centimeters. The factor of two is independent of stiffness, so softening improves signal-to-noise without changing the formula.

3) *Momentum-Observer Estimator*: The two methods above are quasi-static; the momentum observer instead integrates the arm’s dynamics over time and never requires the arm to be at rest. Define the generalized momentum

$$p = B(q) \dot{q}. \quad (10)$$

Differentiating (10) and eliminating $\ddot{q}$ with (2), using the rigid-body identity $\dot{B} = C + C^T$, gives

$$\dot{p} = \tau + \tau_{\text{ext}} + C^T(q, \dot{q}) \dot{q} - g(q), \quad (11)$$

in which the joint acceleration no longer appears, which makes the method noise-robust: no velocity signal is ever differentiated. Following De Luca and Mattone [3], an observer

propagates a predicted momentum $\hat{p}$ from (11) with τ_{ext} replaced by a feedback residual $r = K_O(p - \hat{p})$, $K_O > 0$. Subtracting predicted from true momentum gives

$$\dot{r} = K_O (\tau_{\text{ext}} - r), \quad (12)$$

a first-order low-pass on the external torque: for a static payload, $r \rightarrow \tau_{\text{ext}}$ with time constant $1/K_O$. We run (12) by forward Euler at 500 Hz with $K_O = 50 \text{ s}^{-1}$ ($K_O \Delta t = 0.1$, well within the stability limit; time constant 20 ms). The simulator exposes only the combined bias $C\dot{q} + g$; substituting $\dot{B} = C + C^T$ again avoids separating them, and the $\dot{B}\dot{q}$ term is recovered by finite-differencing the inertia matrix between ticks, keeping the recursion exact even during motion.

After a 0.5 s burn-in (25 time constants) is discarded, the converged residual is regressed against the payload gravity model (3). With the calibrated empty-arm residual baseline r_{empty} subtracted, the least-squares solution over the sampling window is

$$\hat{m} = - \frac{\langle \bar{r} - r_{\text{empty}}, J_z \rangle}{g \|J_z\|^2}, \quad (13)$$

with $\langle \cdot, \cdot \rangle$ the dot product over the six joints. If the pose moves during sampling, the fit is solved sample-wise with the regressor low-passed at the same gain, reducing to (13) at a fixed pose. Calibration is still needed: gravity-model mismatch, Euler drift, and a residual Coriolis term all appear as a constant offset in r that the baseline subtraction removes. The observer needs no gain overrides or gravity mask, and it is the only method that produces a full time profile $r(t)$ rather than a single equilibrium reading.

4) *Inverse-Dynamics Residual Estimator*: The inverse-dynamics estimator treats the payload as an unknown external wrench acting at the end-effector. First, the known robot dynamics are subtracted from the measured joint torque:

$$\tau_{\text{resid}} = \tau - b(q, \dot{q}), \quad (14)$$

where $b(q, \dot{q})$ is the model-predicted bias torque, including gravity and velocity-dependent terms.

For a payload mass m , the residual torque is approximately the joint-space gravity torque induced by the payload:

$$\tau_{\text{resid}} \approx m J_v(q)^T \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = m a(q). \quad (15)$$

Therefore, the payload mass can be estimated by projecting the residual torque onto the payload gravity regressor of (15):

$$\hat{m} = \frac{a(q)^T \tau_{\text{resid}}}{a(q)^T a(q)}. \quad (16)$$

The estimator uses the full multi-joint residual, and because the nominal dynamics are subtracted explicitly it needs no empty-gripper calibration. Samples with large joint velocity or tracking error are gated out, so the fit (16) accumulates only quasi-static evidence and can weigh during a stationary hold or gentle motion.

B. Trajectory Tracking

Accurate trajectory tracking is needed for the pick-and-place motion. We first plan reference trajectories, then track them with two joint-space controllers: a PD controller with gravity compensation (PD+G) and an inverse dynamics (computed-torque) controller.

1) *Trajectory Planning*: Four task-space via points are defined, grasp, weigh, ready-to-release, and release, with the ready-to-release point smoothing the transition into the release motion. These are converted to joint space by analytical inverse kinematics (IK). The IK solver reimplements the closed-form ROS-Industrial `ur_kinematics` formulation [10] in Python, adapted to the MuJoCo UR5e: candidate branch solutions are filtered by joint limits, validated by forward kinematics, and the one closest to the previous configuration is chosen.

Each motion segment is then generated with the Linear Segment with Parabolic Blend (LSPB) method [6], giving position, velocity, and acceleration references for both controllers. Because the target bin depends on the estimated mass, the trajectory is planned in two parts: the pick-and-weigh part is preplanned, the robot holds at the weigh pose for estimation, and the release trajectory is then replanned from the current joint position to the chosen bin. This avoids reference jumps and large transient inputs at the phase transition.

2) *PD With Gravity Compensation Controller*: The PD+G controller is a simple baseline with classical setpoint stability [5]: it uses feedback plus gravity compensation but does not cancel the full nonlinear dynamics, so it is expected to be more sensitive to fast motion. With tracking errors $e = q_d - q$ and $\dot{e} = \dot{q}_d - \dot{q}$, the control law is

$$\tau = K_p e + K_d \dot{e} + g(q) + \tau_{\text{payload}}, \quad (17)$$

where K_p and K_d are diagonal gain matrices and τ_{payload} is the payload compensation defined in (18) below.

Gravity compensation is included in the baseline so the comparison isolates the benefit of full dynamics compensation, rather than being dominated by gravity.

Payload compensation is added to both controllers once the mass is estimated (it is unknown and uncompensated before weighing). After weighing, the feedforward term

$$\tau_{\text{payload}} = J_v(q)^T \begin{bmatrix} 0 \\ 0 \\ \hat{m}g \end{bmatrix}, \quad (18)$$

is added; it maps the estimated payload weight at the end-effector into joint torques, reducing the feedback burden and keeping tracking consistent across payloads.

At release, the payload feedforward is ramped linearly to zero over the release hold rather than dropped in a single step, and the released object is moved clear of the gripper before its collisions are re-enabled. This keeps the commanded torque within the actuator limits through release (Section VI-C).

3) *Inverse Dynamics Controller*: The inverse dynamics controller cancels the inertial, velocity-dependent, and gravitational terms using the full dynamics model [6]. It forms a stabilized reference acceleration and maps it through the model:

$$\tau = B(q) (\ddot{q}_d + K_d \dot{e} + K_p e) + C(q, \dot{q}) \dot{q} + g(q) + \tau_{\text{payload}}. \quad (19)$$

This should track better because the nonlinear dynamics are cancelled explicitly, but it relies on accurate model parameters. In simulation we assume the MuJoCo model supplies an accurate mass matrix and bias terms, so the controller can lean on model-based compensation. As with the PD+G controller, payload compensation (18) is added after weighing to offset the carried object's weight.

VI. RESULTS

A. End-to-End Mission

Fig. 3 shows one complete mission of the weighing pipeline with a 1 kg cube and the sPID estimator: the arm locates the cube on the conveyor, grasps it, lifts to the weigh pose, holds for the one-second weighing window, then follows the planned trajectory to the heavy bin and places the cube. The estimator returned $\hat{m} = 1.006$ kg (+0.6% error), and the classifier routed the cube correctly.

B. Estimators

The four estimators were first characterized in the weighing pipeline, where the cube is physically grasped. Each estimator weighed 20 geometrically spaced masses from 10 g to 3 kg (80 trials total), with calibration performed once at the shared weigh pose. Fig. 4(a) shows the response curves against the ideal $\hat{m} = m$ line and Fig. 4(b) the absolute relative error against requirement E1; Table I summarizes the error statistics.

TABLE I
ESTIMATION ERROR OVER THE 10 G–3 KG SWEEP (PERCENT OF TRUE MASS)

Estimator	Mean err.	Mean err	RMSE
sPID	−2.4 %	2.4 %	2.5 %
Energy-balance	+50.2 %	52.8 %	114.5 %
Momentum observer	+0.8 %	0.8 %	0.9 %
Inverse dynamics	+6.5 %	10.9 %	18.3 %

Figure 4 and Table I support four observations. First, the momentum observer is the most accurate (0.9% RMSE) and the only method that meets E1 over the full range, because it alone combines an empty-arm baseline with a six-joint residual, so neither model bias nor single-channel noise limits it. Second, the methods separate at the *light* end: each carries a roughly constant few-gram absolute error, so the relative error grows as $1/m$ as the payload shrinks. This is clearest for the calibration-free inverse-dynamics estimator (Fig. 4(b)), whose relative error falls nearly two decades from 10 g to 3 kg. Third, calibration trades convenience for light-mass accuracy: the calibration-free inverse-dynamics estimator inherits the gripper-linkage torques as a constant bias (18.3% RMSE),

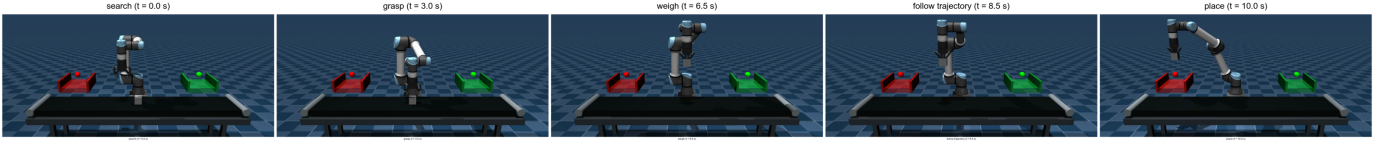


Fig. 3. Time-lapse of one mission (1 kg cube, sPID estimator): search, grasp at the conveyor, weigh hold, trajectory following to the bin, and placement into the heavy bin.

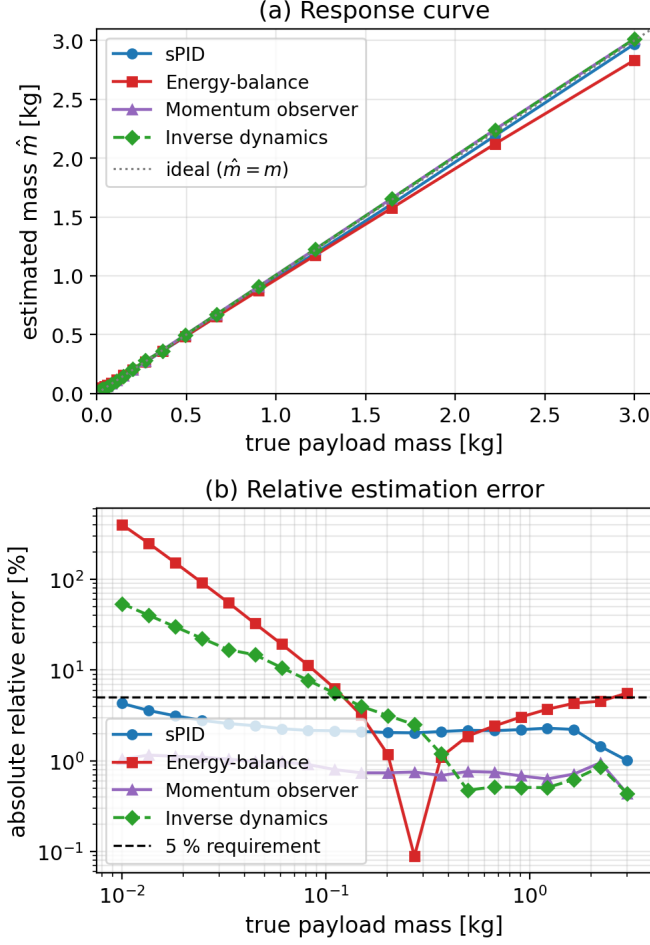


Fig. 4. Standalone estimator accuracy over the 10 g to 3 kg sweep (20 masses per estimator, weighing pipeline). (a) Estimated versus true mass with the ideal line. (b) Absolute relative error on log-log axes with the 5% requirement E1 (dashed).

whereas the calibrated sPID cancels the same bias and stays within 2.5% using only one joint. Fourth, the energy-balance estimator fails on light payloads (114.5% RMSE), which answers the question raised at our presentation: both the numerator ΔE_{spring} and denominator Δh in (9) shrink with the payload (energy quadratically, sag linearly), so below about 0.1 kg the sag is lost in settling noise and the small-over-small quotient is unreliable. Above about 0.5 kg it tracks within $\pm 10\%$.

C. Controllers

Both controllers' gains are tuned to satisfy C1 (joint error within $\pm 2^\circ$) and C2 (commanded torque below 0.95 of the limits) from Section III.

1) *Effect of Payload Compensation on Tracking Performance:* The overall tracking performance of the two controllers is first evaluated. Fig. 5 shows the joint tracking error time histories when the robot carries a 1.5 kg payload through the full mission. Both controllers generally satisfy the tracking accuracy requirement: while the cube is carried without compensation (between the grasp at $t \approx 3$ s and the end of the weighing window at $t \approx 7.5$ s) the wrist-1 and elbow errors plateau at $0.2\text{--}0.35^\circ$, and once the estimated-mass feedforward (18) engages, the errors collapse to nearly zero. This improvement remains effective until the object is released.

The commanded torque stays within the actuator limit through release. Its peak over the *entire* mission, including grasp and release, is 0.35 times the actuator limit for both controllers and occurs during the carrying motion ($t \approx 12$ s) rather than at release. A small error bump appears at release ($t \approx 14$ s in Fig. 5) but stays well within the $\pm 2^\circ$ accuracy requirement. The controller comparison below therefore spans the full carrying interval through release, with no part of the motion excluded.

2) *Payload Robustness:* Payload mass and motion speed were the two factors that most affected performance during tuning, so we evaluate robustness to each; each controller keeps its own fixed gains, scored on C1 and C2. Payload robustness is tested at a fixed motion speed (move-to-release time 4.0 s) with the payload varied from 0 to 3 kg (MuJoCo forbids exactly zero mass, so a tiny mass approximates 0 kg). True mass is used for compensation here, to isolate controller behavior from estimator error.

Fig. 6 shows the peak commanded torque, normalized by the per-joint actuator limit, against payload mass. It rises roughly linearly from about 0.29 at near-zero payload to about 0.42 at 3 kg for both controllers, staying far below the 0.95 feasibility limit (C2) across the whole range. The two controllers are nearly indistinguishable here because the dominant torque is the shared gravity-and-payload feedforward, not the feedback term.

Fig. 7 shows the peak joint position error against payload mass. It grows modestly with mass but stays far below the 2° requirement (C1) for both controllers across the whole range. The inverse dynamics controller is nearly exact at light payloads (under 0.1° , where its model feedforward leaves

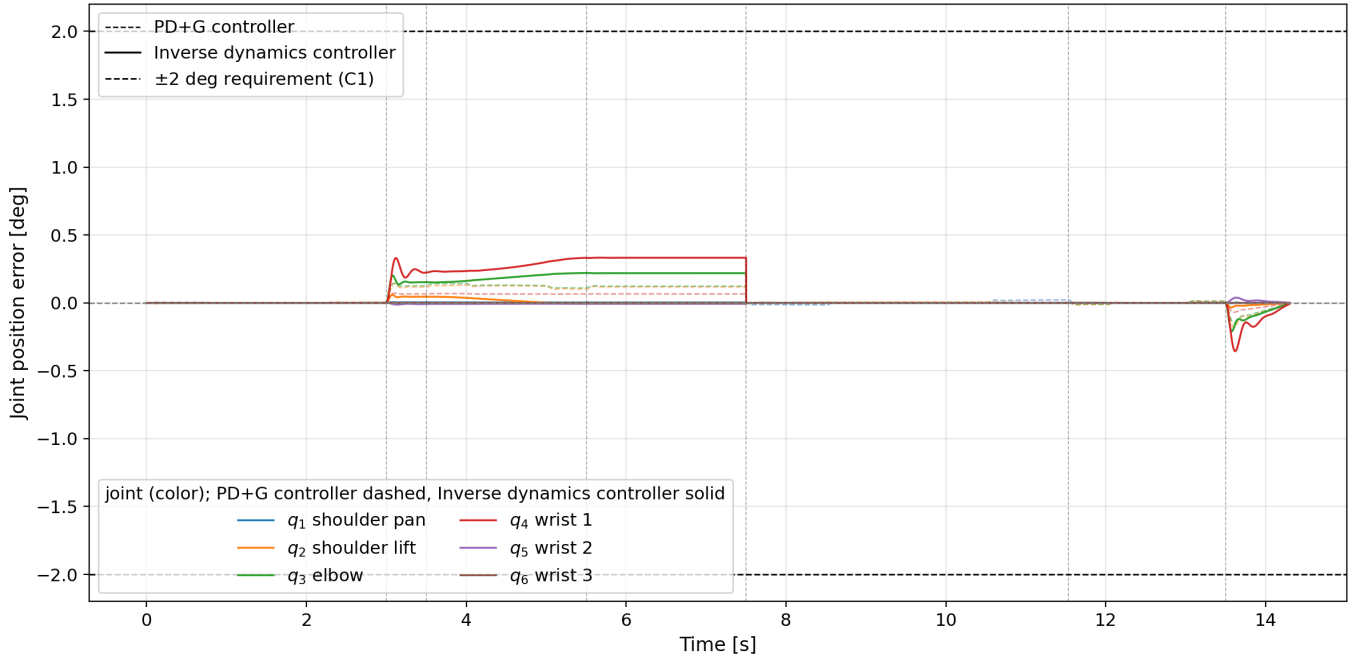


Fig. 5. Joint tracking error time histories for the full mission with a 1.5 kg payload: PD+G controller (dashed) versus inverse dynamics controller (solid), with one color per joint as labeled in Fig. 1. The horizontal dashed lines mark zero error and the $\pm 2^\circ$ requirement C1; vertical lines mark trajectory stage changes. Payload compensation engages after the weigh window ($t \approx 7.5$ s); at release ($t \approx 14$ s) the commanded torque stays within C2, leaving only a small error bump well inside the $\pm 2^\circ$ requirement.

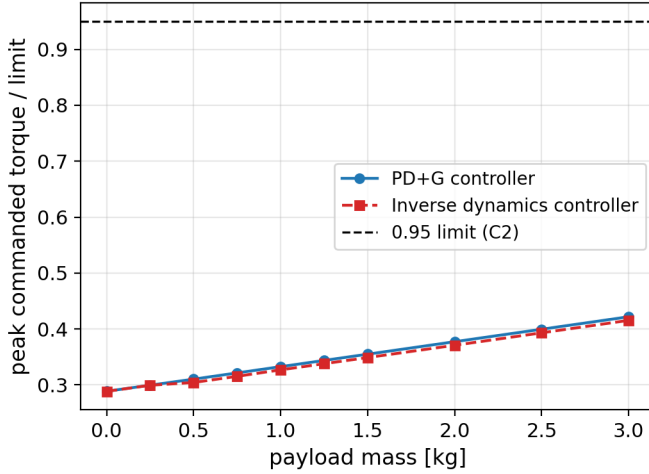


Fig. 6. Peak commanded torque (as a fraction of the per-joint actuator limit) versus payload mass, at a fixed 4.0 s move-to-release time. The dashed line is the 0.95 feasibility limit (C2). Markers show the sampled payloads.

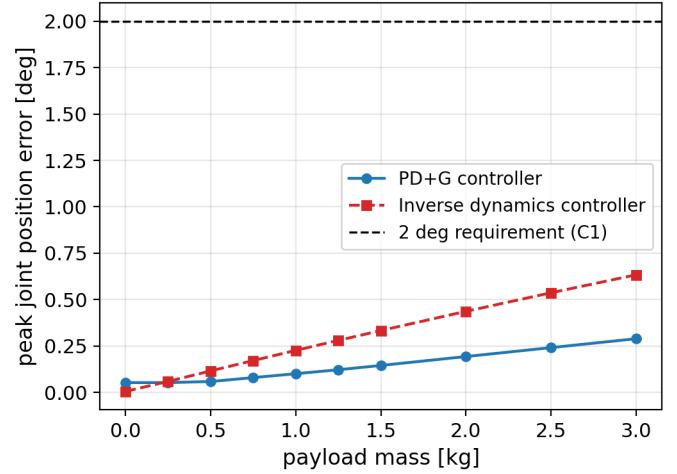


Fig. 7. Peak joint position error versus payload mass, at a fixed 4.0 s move-to-release time. The dashed line is the $\pm 2^\circ$ tracking requirement (C1). Markers show the sampled payloads.

almost no residual error) but grows to about 0.6° at 3 kg, while the PD+G controller stays between about 0.05° and 0.3° ; the two cross near 0.6 kg. The difference is small in absolute terms, since both remain an order of magnitude inside the requirement.

Overall, both controllers are robust to payload from 0 to 3 kg at this speed: the payload feedforward keeps the torque demand and tracking error low and nearly flat, and both stay

far inside the requirements regardless of mass.

3) *Speed Robustness*: Speed robustness is evaluated at a fixed 1.5 kg payload, varying the move-to-release time; a shorter time means a faster motion, so the aggressive regime is on the left of each plot.

Fig. 8 shows the peak torque ratio against move-to-release time. For move times of 1 s or longer both controllers sit between about 0.35 and 0.5, but as the motion is compressed

the required torque rises steeply, and at the shortest tested time (0.5 s) both controllers exceed the 0.95 limit (ratios of about 2.8 for PD+G and 2.0 for inverse dynamics). The PD+G controller reaches the limit slightly sooner, crossing 0.95 near a 0.6 s move time.

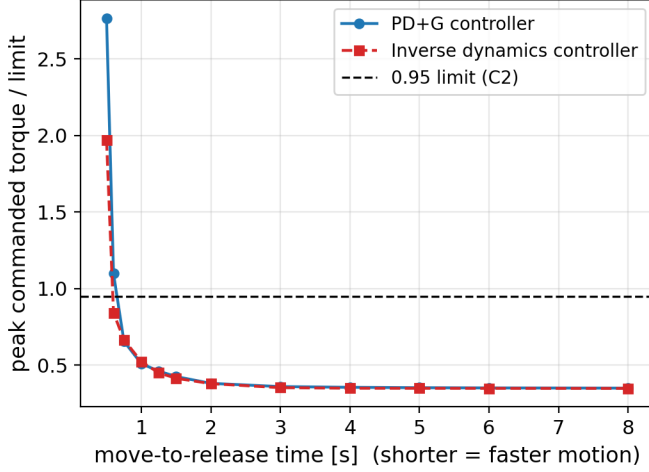


Fig. 8. Peak commanded torque (as a fraction of the actuator limit) versus move-to-release time, at a fixed 1.5 kg payload. Shorter time is faster motion. The dashed line is the 0.95 feasibility limit (C2).

Fig. 9 shows the peak joint error against move-to-release time, and this is where the two controllers separate most clearly. The inverse dynamics controller holds the error near 0.33° for every move time down to 0.6 s, exceeding the 2° requirement only at the most aggressive 0.5 s setting. The PD+G controller degrades much earlier: its error crosses 2° near a 0.7 s move time and reaches 6.3° at 0.5 s. The reason is structural: the PD+G law has no inertial feedforward, so it must develop a growing tracking error to generate the accelerating torques a fast trajectory demands, whereas the inverse dynamics controller supplies those torques from the model.

These results show that motion speed has a stronger influence on controller performance than payload mass in this task. With payload compensation, both controllers are robust to payload changes. However, for faster trajectories, the inverse dynamics controller performs better because it directly compensates for inertial, velocity-dependent, and gravitational dynamics. Therefore, the inverse dynamics controller is more suitable for maintaining both tracking accuracy and torque feasibility under faster motion conditions.

D. Integration

We then ran the full pipeline, pairing each of three estimators (sPID, inverse-dynamics, momentum observer) with each of the two tracking controllers; the energy-balance estimator was dropped here given its poor light-mass accuracy in the standalone study (Section VI-B). Each trial ran the pick-and-place trajectory, then held at a stationary weighing pose where the estimator inferred the mass. The question was whether a

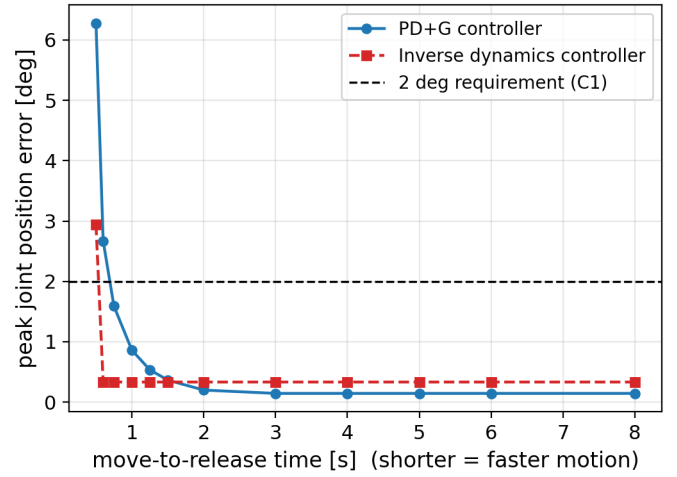


Fig. 9. Peak joint position error versus move-to-release time, at a fixed 1.5 kg payload. Shorter time is faster motion. The dashed line is the $\pm 2^\circ$ tracking requirement (C1).

better tracking controller, or a controller-estimator pair with matching principles, gives a better estimate. The payload was swept geometrically from 10 g to 3 kg (16 masses), matching the standalone study, and each estimate is compared with ground truth as an absolute relative error in Fig. 10, one panel per controller.

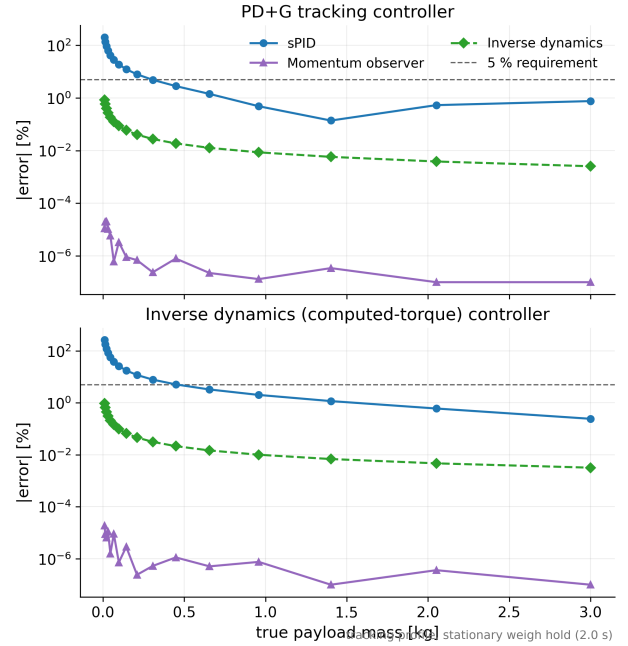


Fig. 10. Absolute relative payload estimation error over the 10 g–3 kg sweep, for each controller-estimator pair in the tracking (force-injection) pipeline. In this pipeline the payload is applied as a known force at the end-effector, so the momentum observer and inverse-dynamics estimators recover it essentially exactly; their curves sit at the floating-point noise floor and should be read as “at machine precision,” not as a meaningful ranking between the two.

1) *Assessment of the Results:* We expected controller-estimator pairs with matching principles to do best, e.g. PD

tracking with sPID (the payload shows up in the feedback torque), or inverse dynamics control with the inverse-dynamics estimator (both reason about model-based torque).

The results need care to interpret. In the tracking pipeline the payload is injected as a known force $[0, 0, -mg]$ at the pinch site, and the residual-based estimators recover that same force through the same Jacobian, so the momentum observer ($\approx 10^{-5}$ % RMSE) and the inverse-dynamics estimator (0.2–0.3 % RMSE) sit at the numerical noise floor of the simulation rather than being meaningfully separated (Fig. 10). The sPID method reads a single-joint controller signal rather than the injected force, so it is the only estimator here whose error reflects controller behavior. It shows the largest error, worse under the inverse-dynamics controller (89.8 % RMSE) than under the PD+G controller (68.0 % RMSE), as a systematic, mostly negative bias; both figures are dominated by the light end of the sweep, where a fixed few-gram error is a large fraction of the true mass.

The residual-based estimators (inverse dynamics and momentum observer) read payload-induced dynamics residuals rather than controller error, so they barely depend on the controller once the arm reaches a stable weighing pose; the controller only affects how well that pose is held, not the final estimate. The sPID estimator instead reads a single-joint torque/error signal. Because the payload gravity torque is actually spread across several joints, this single-joint reading also picks up tracking error, settling transients, and joint coupling, which makes it controller-sensitive and the least accurate of the three here.

The inverse-dynamics estimator also does far better here than in the standalone study (Fig. 10 vs. Fig. 4(b)), because of the payload model. The tracking pipeline applies a pure gravity force, so the model bias $b(q, \dot{q})$ in (14) is exact and the calibration-free residual is clean; in the grasp-based pipeline the gripper linkage adds steady torques that the uncalibrated estimator absorbs as error. This shows how much the “no calibration needed” property depends on payload- and gripper-model fidelity.

The force-injection result isolates the controller cleanly, but it also flatters the residual estimators, which recover the injected force almost exactly. To see how the same three estimators behave against real contact, we re-ran the integration sweep in the physically grasped pipeline of Section IV, with the cube held by the gripper. Grasping and gross motion use the PD law in every trial, and only the control law holding the weigh pose is changed; the grasp is therefore identical across trials and the comparison isolates the controller’s effect on the estimate itself (Fig. 11, Table II).

In this setting the residual estimators no longer sit at machine precision. Under the PD+G hold the momentum observer holds 0.9 % RMSE and the inverse-dynamics estimator 18.8 %, reproducing the standalone numbers of Table I and recovering the accuracy ranking that Fig. 10 cannot show. Over the range that matters for classification, near and above the 0.35 kg threshold, the estimate barely changes between the two control laws, consistent with the equilibrium argument above; the

controller-induced spread is confined to the lightest masses. The sPID estimator is the exception, with its RMSE rising from 2.5 % under PD+G to 72.3 % under computed-torque. This confirms that the single-joint reading is the one genuinely sensitive to the controller, while the residual estimators are not.

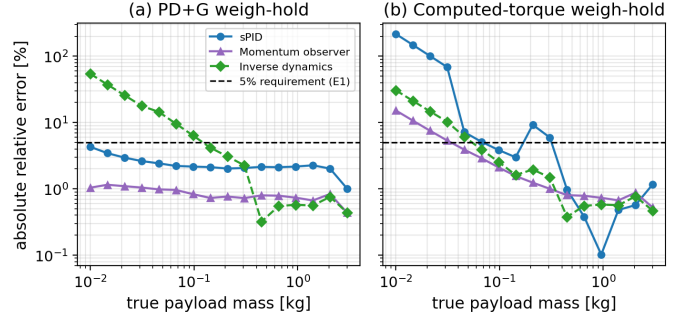


Fig. 11. Absolute relative estimation error over the 10 g–3 kg sweep for the three estimators in the *physically grasped* pipeline, with the weigh pose held under (a) the PD+G law and (b) the computed-torque law. Unlike the force-injection figure (Fig. 10), the residual estimators sit at their true accuracy rather than the numerical floor. The dashed line is the 5 % requirement E1.

TABLE II
GRASP-PIPELINE INTEGRATION ERROR OVER THE 10 G–3 KG SWEEP
(PERCENT OF TRUE MASS), WITH THE WEIGH POSE HELD UNDER EACH
CONTROL LAW

Weigh-hold law	Estimator	Mean err.	RMSE
PD+G	sPID	2.4 %	2.5 %
PD+G	Momentum observer	0.8 %	0.9 %
PD+G	Inverse dynamics	11.1 %	18.8 %
Computed-torque	sPID	35.7 %	72.3 %
Computed-torque	Momentum observer	3.5 %	5.4 %
Computed-torque	Inverse dynamics	6.1 %	10.5 %

Overall, the inverse dynamics controller tracks better and is more stable, and both residual estimators are accurate, so pairing inverse dynamics control with either the inverse-dynamics or momentum-observer estimator suits this task.

There are several limitations. Because the force-injection pipeline drives the residual-based estimators to machine precision, the ranking between the momentum observer and the inverse-dynamics estimator comes instead from the physically grasped runs (Table II and the standalone study of Section VI-B), where the gripper-linkage torques expose their real differences. More broadly, the simulation assumes accurate joint states, accurate model terms, clean torque information, and simplified gripper-payload interaction, all of which make residual-based estimation easier than it would be on real hardware.

VII. DISCUSSION

A. Gain Choices for the Trajectory-Tracking Controllers

The best gains depend on payload and speed, so a single nominal condition, a 1.5 kg payload with a 4.0 s move-to-release time, is fixed for a fair comparison. Starting from the pre-tuned position-control PID gains shipped with the UR5e

model, we tune the gains to satisfy both C1 and C2 at this condition without becoming so aggressive that they only work there.

The most useful way to read the two gain sets is through the closed-loop error dynamics they produce, not the raw numbers. For the inverse dynamics controller, the computed-torque law (19) cancels $B(q)$ and the nonlinear terms, so each joint's tracking error $e = q_d - q$ obeys a decoupled linear second-order system,

$$\ddot{e} + K_d \dot{e} + K_p e = 0, \quad (20)$$

with natural frequency and damping ratio

$$\omega_n = \sqrt{K_p}, \quad \zeta = \frac{K_d}{2\sqrt{K_p}}. \quad (21)$$

The chosen per-joint gains are

$$K_p = [1800, 2200, 1800, 720, 560, 400] \text{ N m/rad}, \quad (22)$$

$$K_d = [28, 32, 28, 14, 12, 10] \text{ N m s/rad}, \quad (23)$$

which by (21) give $\omega_n \approx 42\text{--}47$ rad/s on the three proximal joints and $20\text{--}27$ rad/s at the wrists, with $\zeta \approx 0.33$ proximally and ≈ 0.25 at the wrists, so the loops are underdamped. A step input to (20) at $\zeta = 0.33$ would overshoot about 33%, but the LSPB references are smooth, so the tracking error shows no overshoot or ringing (Fig. 5). The low K_d keeps the velocity-feedback torque, and hence the peak commanded torque, within C2.

For the PD-with-gravity-compensation controller (17), $B(q)$ is *not* cancelled, so the error dynamics are $B(q)\ddot{e} + K_d\dot{e} + K_p e \approx 0$ and the gains act through the joint inertia. The chosen gains are

$$K_p = [3150, 3850, 3150, 1260, 980, 700] \text{ N m/rad}, \quad (24)$$

$$K_d = [78.4, 89.6, 78.4, 39.2, 33.6, 28.0] \text{ N m s/rad}. \quad (25)$$

Dividing the gains (24)–(25) by the mass-matrix diagonal at the weigh pose ($B_{ii} \approx [1.26, 2.48, 1.03, 0.16, 0.13, 0.10] \text{ kg m}^2$) gives effective frequencies $\omega_n = \sqrt{K_p/B_{ii}} \approx 39\text{--}90$ rad/s and damping $\zeta = K_d/(2\sqrt{K_p B_{ii}}) \approx 0.5\text{--}0.7$ on the proximal joints, rising above one (overdamped) at the light wrist joints. This is why the larger raw K_p of the PD+G set is not directly comparable to the inverse dynamics set: most of the apparent difference is the factor of B_{ii} that the inverse dynamics controller has already divided out. The practical consequence is that the PD+G loop has no model feedforward to lean on, so it must raise the commanded torque steeply to hold tracking as the motion speeds up, which is exactly the speed sensitivity seen in Section VI-C.

B. Gain Choices for the Estimators

Each estimator needed its own gain choice. For sPID, the base stiffness ($k_p \approx 2000$) at the uncompensated elbow caused a limit cycle of over 100 N·m peak-to-peak; softening to $k_p = 200$, $k_d = 80$ removed it and changed only the variance, not the steady-state torque or the estimate (6). For

the energy-balance method, softening k_p tenfold while halving k_d kept the damping regime so the factor of two in (8) stayed valid; softening k_d as much as k_p instead made the loop underdamped and biased the estimate upward. For the momentum observer, K_O trades convergence speed against noise filtering, and $K_O = 50 \text{ s}^{-1}$ converges within about 100 ms. None of these choices interact with the tracking-controller gains, which is what lets the estimators be swapped freely.

C. Capabilities and Limitations

When pushed beyond the conditions of Section VI-C, each component fails in its own way. The PD+G controller is limited by speed. In Fig. 9 its peak joint error crosses the 2° requirement when the move-to-release time drops below roughly 0.7 s and reaches 6.3° at the fastest tested motion (0.5 s), while its torque ratio climbs past the 0.95 limit near a 0.6 s move time (Fig. 8). The inverse dynamics controller stays within both requirements over nearly the entire tested range, holding about 0.33° down to a 0.6 s move time and breaking only at the most aggressive 0.5 s setting. Against payload, neither controller violates a requirement within 0–3 kg once payload compensation is active (Figs. 6, 7); extrapolating the linear torque trend, the shoulder actuators would saturate near 10 kg, far beyond the physical payload rating.

Two qualifications temper this controller comparison. The inverse dynamics law is evaluated against the same model it uses for compensation (assumption (i)), so its advantage at speed is an upper bound: with model error the computed-torque cancellation would be imperfect and the margin over PD+G would narrow. The advantage is also concentrated near the torque-feasibility edge. For the slower move times where both controllers satisfy C2 their tracking is nearly identical, and the inverse dynamics controller pulls clearly ahead only around 0.5–0.7 s, where the commanded torque is itself approaching or exceeding the C2 limit.

On the estimator side, the breaking points are all at the light end. The energy-balance method is unusable below about 0.1 kg, where its signal vanishes; the calibration-free inverse-dynamics method degrades below a few hundred grams in the grasp-based pipeline, where the unmodeled gripper torque is a constant absolute error; and sPID holds to 2.5% RMSE but drifts slowly as heavier cubes move the loaded pose away from the calibrated one, shifting the moment arm in (6). The momentum observer's weakness is geometric, not mass-dependent: a weigh pose with $\|J_z\| \approx 0$ would make the projection (13) undefined, but the chosen pose keeps $\|J_z\|^2 \approx 0.4$, far from that. At release, the commanded torque stays within C2 across the whole mission, leaving only a small error bump inside C1. One caveat applies to every number in this report: the simulation provides noise-free joint states and exact model terms, so the reported tracking and estimation accuracies are best cases, and all four estimators in particular should be expected to lose accuracy once real encoder noise, torque ripple, and friction are present.

D. Challenges

We ran into three problems we had not planned for. Early on, a limit cycle at the sPID joint left estimates correct on average but far too noisy. Once we traced the noise to the limit cycle rather than to the estimator itself, we added the per-estimator gain overrides that all four methods now share. The energy-balance estimator took longer to sort out: it returned exactly half the true mass for weeks, and the missing factor of two turned out to be the damper’s dissipation (Section V-A2), which the energy balance has to count alongside the spring. The last problem came from the gripper. Its spring linkage applies steady wrist torques that the gravity model does not include, and these corrupted every uncalibrated residual until the empty-arm calibration absorbed them. They are still the main error source for the calibration-free estimator when the cube is physically grasped.

VIII. FUTURE DIRECTIONS

Several extensions follow from the limitations above. (i) *Wider payload range*: sweeping beyond 3 kg toward actuator saturation would separate the momentum observer and inverse-dynamics estimators and stress payload compensation near the limits. (ii) *Physical grasp during in-motion weighing*: with a grasped object in motion, the regressor in (13) must add the payload’s inertial reaction $J_v^T(a_{ee} + g\hat{z})$ (with a_{ee} the end-effector acceleration), and contact compliance, grasp offset, and slip must be modeled. (iii) *Pipeline and perception*: optimizing the weigh pose ($\max\|J_z\|$), shortening the hold using the observer’s convergence diagnostic, and swapping ground-truth perception for the camera backend would shorten the cycle. (iv) *Learned estimation*: the four estimators all consume the same per-tick observation (joint state, commanded and measured torque, gravity bias, inertia matrix, and end-effector Jacobian), which is also a natural input to a learned model. A reinforcement-learning agent or a supervised regressor trained on this joint-signal stream could output a mass prediction directly, potentially absorbing the unmodeled effects (gripper-linkage torques, friction, contact) that limit the analytical estimators, at the cost of training data and interpretability. Hardware validation is the most important next step: simulation’s noise-free sensing and exact models flatter all four estimators, and the open question is how much accuracy survives real encoders, torque ripple, and friction.

Beyond these specific extensions, the broader potential of the approach is to make a dedicated weighing station unnecessary on any torque-controlled arm. Because the estimators read only signals the controller already produces, the same software could add in-place weighing to existing pick-and-place cells at no extra hardware cost, supporting weight-based sorting, adaptive grasp force, and simple quality checks that flag items whose measured mass departs from the expected value. Because the momentum observer produces a continuous residual, the same signal also provides a low-bandwidth estimate of external force, which could be shared with collision detection and safer human-robot interaction.

The approach is not limited to gravity-based weighing on the ground. In microgravity a conventional scale does not work, since there is no weight to read, so estimating mass from the manipulator’s own dynamics may be the only practical option. The gravity-based estimators here would not carry over directly, but the inertial-reaction variant of item (ii) reads mass from the $J_v^T a_{ee}$ term during commanded accelerations, which is the quantity that survives when gravity does not. That is the relevant case for on-orbit servicing and debris removal, where knowing the mass of a captured object before maneuvering it improves both safety and fuel use. More generally, because the estimators use only signals the controller already computes, in-place mass estimation is better understood as a capability that any model-based manipulator can add than as a feature of one sorting cell.

IX. CONCLUSION

We built a simulated mass-aware sorting cell that weighs objects with no scale and no force/torque sensor, and used it to compare four estimators and two controllers under common, quantified requirements. The recommended estimator is the momentum observer: it meets the 5% accuracy requirement over the full 10 g–3 kg range (0.9% RMSE), is independent of the tracking controller, and produces a continuous estimate rather than a single equilibrium reading. The recommended controller is the inverse dynamics law (19) with the gains (22)–(23) ($\omega_n \approx 42$ rad/s, $\zeta \approx 0.33$ on the proximal joints): at slow speed it only slightly beats the PD+G baseline, but it holds the $\pm 2^\circ$ tracking and 95% torque requirements at speeds where PD+G has already failed, because it cancels the inertial and velocity-dependent dynamics PD+G leaves to feedback. This advantage is measured against an exact dynamics model and would narrow under model error. Using the estimated-mass feedforward of (18), this pair ran the full pick-weigh-sort mission across the tested range of payloads. In simulation it stayed within both tracking requirements, and with the momentum observer it also met both estimation requirements.

ACKNOWLEDGMENT

We used generative AI coding tools (Claude Code by Anthropic and Codex by OpenAI) during software development, under our direction, to implement and debug portions of the code and to write test, plotting, and figure-generation scripts (see Appendix B); the system architecture, the derivation of the four estimators, the controller designs, the experimental design, and all gain tuning were our own. We also used Claude to help produce the diagrams and graphs in this paper and to proofread the manuscript and check grammar and wording. We reviewed all AI-assisted code and text, and validated all results through the test suite and the experiments reported in this paper.

APPENDIX A TEAM MEMBER CONTRIBUTIONS

All members took part in the controller design discussions and in writing this report. The specific contributions were:

- **Yonghao Huo:** contributed to project ideation and to early design discussions of the inverse dynamics controller and the trajectory-tracking approach (Section V-B).
- **Lisha Yuan:** selected the robot and gripper models and built the simulation scene (Section IV); implemented the inverse dynamics controller, the inverse-dynamics residual estimator, and the trajectory-tracking pipeline.
- **Han Wu:** helped build the simulation scene (Section IV); implemented the inverse dynamics controller, PD-with-gravity-compensation controller, the inverse-dynamics residual estimator, and the trajectory-tracking pipeline.
- **Zejun Xiong:** designed the codebase architecture and the shared estimator interface; wrote the MuJoCo simulation environment code; designed and implemented the sPID, energy-balance, and momentum-observer estimators (Sections V-A1–V-A3) and implemented the PD-with-gravity-compensation controller.

APPENDIX B CODE AND MATERIALS

All software is custom Python written for this project: the MuJoCo scene and mission pipelines, the four estimators behind a common interface, the two tracking controllers, the analytical IK, and the sweep/plotting scripts that produced every figure in this report. The code accompanies this report in the project submission folder and is available at [11].

We designed and architected the codebase before introducing AI coding tools. We specified the layered architecture, the estimator interface and calibration protocol, the finite-state machine, and the controller structures, and derived the estimator math by hand. Claude Code (Anthropic) and Codex (OpenAI) were then used to implement details within that design, generate unit tests and plotting scripts, and assist with debugging. We reviewed every AI-assisted file and checked correctness against the analytical derivations (for example, the energy balance of Section V-A2) and the project’s test suite.

No bill of materials applies, as the project is entirely simulation-based; the UR5e and Robotiq 2F-85 models are open-source assets [9] and are credited in Section IV.

REFERENCES

- [1] N. Correll, K. E. Bekris, D. Berenson, O. Brock, A. Causo, K. Hauser, K. Okada, A. Rodriguez, J. M. Romano, and P. R. Wurman, “Analysis and observations from the first Amazon Picking Challenge,” *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 1, pp. 172–188, 2018.
- [2] C. G. Atkeson, C. H. An, and J. M. Hollerbach, “Estimation of inertial parameters of manipulator loads and links,” *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 101–119, 1986.
- [3] A. De Luca and R. Mattone, “Sensorless robot collision detection and hybrid force/motion control,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Barcelona, Spain, 2005, pp. 999–1004.
- [4] S. Haddadin, A. De Luca, and A. Albu-Schäffer, “Robot collisions: A survey on detection, isolation, and identification,” *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1292–1312, 2017.
- [5] M. Takegaki and S. Arimoto, “A new feedback method for dynamic control of manipulators,” *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 103, no. 2, pp. 119–125, 1981.
- [6] B. Siciliano, L. Sciacivco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.

- [7] J.-J. E. Slotine and W. Li, “On the adaptive control of robot manipulators,” *The International Journal of Robotics Research*, vol. 6, no. 3, pp. 49–59, 1987.
- [8] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vilamoura, Portugal, 2012, pp. 5026–5033.
- [9] Google DeepMind, “MuJoCo Menagerie: A collection of high-quality models for the MuJoCo physics engine,” GitHub repository, 2022. [Online]. Available: https://github.com/google-deepmind/mujoco_menagerie. Accessed: Jun. 11, 2026.
- [10] ROS-Industrial, “universal_robot: ur_kinematics package (ur_kin.cpp),” GitHub repository, commit on the kinetic-devel branch. [Online]. Available: https://github.com/ros-industrial/universal_robot/blob/kinetic-devel/ur_kinematics/src/ur_kin.cpp. Accessed: Jun. 11, 2026.
- [11] Y. Huo, L. Yuan, H. Wu, and Z. Xiong, “MassAwareArm: course project code for an autonomous mass-aware sorting system,” GitHub repository, 2026. [Online]. Available: <https://github.com/MichaelXiongZJ/MassAwareArm>.